

MySQL Performance Tuning

What is Performance Tuning?

Performance Tuning is the process of making a MySQL database **faster, more efficient, and able to handle more users.**

Real-Life Example

Imagine a supermarket.

- **1 billing counter** → Customers wait longer. ❌
- **5 billing counters** → Customers are served quickly. ✅

Similarly, performance tuning reduces the time required to execute SQL queries.

Why is Performance Tuning Needed?

Without tuning:

- Queries are slow.
- CPU usage is high.
- Memory usage increases.
- Users wait for results.

With tuning:

- Faster query execution.
- Better resource utilization.
- Better user experience.

Example

Employees Table

EmployeeID	Name	Department	Salary
1	Kamalesh	IT	50000
2	Rahul	HR	60000
3	Priya	IT	70000
...	...	...	...

Suppose the table has **10 million rows**.

Query:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT';
```

Without optimization, MySQL may scan **all 10 million rows**.

This is called a **Full Table Scan**.

1. Indexing

What is an Index?

An **Index** is a special data structure that helps MySQL find rows quickly.

Example

Without Index

Book

Page 1

Page 2

Page 3

...

Page 500

To find **Chapter 10**, you search page by page.

With Index

Index

Chapter 10 → Page 230

You go directly to Page 230.

SQL Example

Without Index

```
SELECT *  
FROM Employees  
WHERE Department = 'IT';
```

Create Index

```
CREATE INDEX idx_department  
ON Employees(Department);
```

Now MySQL finds IT employees much faster.

2. Use WHERE Clause

Bad Query

```
SELECT * FROM Employees;
```

Reads every row.

Better Query

```
SELECT *  
FROM Employees  
WHERE EmployeeID = 5;
```

Reads only one row.

3. Avoid SELECT *

Bad

```
SELECT *  
FROM Employees;  
  
Returns all columns.
```

Good

```
SELECT Name, Salary  
FROM Employees;  
  
Fetch only the required columns.
```

4. LIMIT

Bad

```
SELECT *  
FROM Employees;  
  
Returns thousands of rows.
```

Good

```
SELECT *  
FROM Employees  
LIMIT 10;  
  
Only 10 rows are returned.
```

5. EXPLAIN

Use EXPLAIN to see how MySQL executes a query.

Example

EXPLAIN

SELECT *

FROM Employees

WHERE Department = 'IT';

Output (simplified)

id	table	type	key
1	Employees	ALL	NULL

ALL means **Full Table Scan**.

After creating an index:

EXPLAIN

SELECT *

FROM Employees

WHERE Department='IT';

Output

id table type key

1 Employees ref idx_department

Now MySQL uses the index.

6. Optimize JOINS

Bad

```
SELECT *  
FROM Orders  
JOIN Customers;
```

(No join condition)

Produces a huge Cartesian product.

Good

```
SELECT Customers.CustomerName,  
       Orders.Product  
FROM Customers  
JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

7. Use Proper Data Types

Bad

Salary VARCHAR(100)

Salary is numeric, so storing it as text wastes space and slows comparisons.

Good

Salary INT

8. Normalize Database

Bad

Employee Department Manager Department

Duplicate data.

Good

Separate tables

Employees

Departments

Managers

Less redundancy and easier maintenance.

9. Use Transactions Efficiently

Bad

START TRANSACTION;

(wait 5 minutes)

COMMIT;

Long-running transactions hold locks for a long time.

Good

START TRANSACTION;

UPDATE Employees

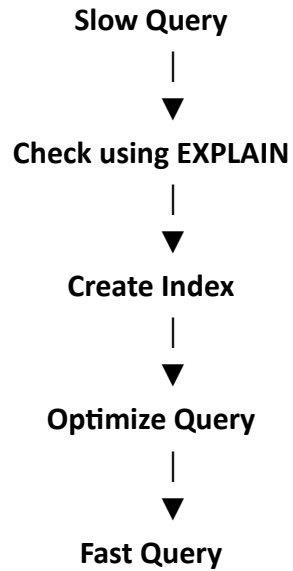
SET Salary = 60000

WHERE EmployeeID = 1;

COMMIT;

Complete the transaction quickly.

Performance Tuning Flow



Example

Without Index

```
SELECT *  
FROM Employees  
WHERE Name='Rahul';
```

Time:

5 seconds

After Index

```
CREATE INDEX idx_name  
ON Employees(Name);
```

Now

```
SELECT *  
FROM Employees  
WHERE Name='Rahul';
```

Time:

0.02 seconds

1. What is Performance Tuning?

Answer:

Performance tuning is the process of improving the speed and efficiency of database operations by optimizing queries, indexes, schema design, and server configuration.

2. What are common Performance Tuning techniques?

- Create indexes
- Avoid SELECT *
- Use WHERE clauses effectively
- Use LIMIT when appropriate
- Analyze queries with EXPLAIN
- Optimize JOIN operations
- Choose proper data types
- Keep transactions short

3. What is EXPLAIN?

EXPLAIN shows the execution plan of a query, helping you understand whether MySQL is using indexes or scanning entire tables.

Example:

```
EXPLAIN
SELECT *
FROM Employees
WHERE EmployeeID = 1;
```

- **P** – Proper Indexes
- **E** – EXPLAIN queries
- **R** – Reduce unnecessary data (SELECT only needed columns)
- **F** – Filter with WHERE
- **O** – Optimize JOINS
- **R** – Right data types
- **M** – Minimize long transactions